Scaffold Counting Model

Test task report for **Kotone**

Author: Christian Ishimwe, Gisenyi, Rwanda. Submitted after an extension

Summary

- The 5 test photos show **stacked steel scaffold planks**, not round tubes. The countable unit is the **plank end-face**. The model also handles tube bundles.
- Approach: a YOLOv8n detector trained **only on synthetic images** (no real labelled data was available), plus a user-selected region (ROI) so that only one stack is counted, as in the reference app.
- Results: counts for all 5 images with detections marked (section 3). The **correct counts have not been provided**, so the reference column is my own visual estimate and is not ground truth.
- Speed: about 15 ms per image on a Colab T4 GPU and about 70 ms on 2 CPU cores (512 px input).
- Status: a working baseline.
- **Live demo (temporary on , only while my Colab notebook runs):** https://94e06bbd94323fad08.gradio.live

1. What the test images contain

The brief says "pipes (tubes)", but all 5 photos show stacked steel scaffold planks seen from the end or the front. I built for planks and kept tube support because the reference app also counts tubes, so the same model serves both.

Scope rule: count the planks of the stack the user selects (a box drawn on the photo). Neighbouring stacks, side faces and background planks are not counted.

2. Approach

1. **Data.** No real labelled data exists and the 5 images are reserved for testing only. `src/synth_data.py` draws synthetic images with exact box labels: plank stacks (1 to 5 columns, 6 to 25 layers, galvanised tones, 0 to 2 bolt holes) and hex tube bundles, with random perspective, backgrounds, exposure, glare, blur and JPEG compression.
2. **Model.** YOLOv8n (COCO pre-trained), fine-tuned at 512 px. Classes: `plank_end` and `tube_end`. Final run (Colab T4): 1,000 train and 150 val synthetic images, 30 epochs. First run (CPU): 300 train, 60 val, 30 epochs. Synthetic validation mAP50 is about 0.995.
3. **Counting.** Detect with confidence >= 0.5 and NMS IoU 0.5, drop boxes wider than 6x their height (side faces), and keep boxes whose centre is inside the ROI.
4. **Classical baselines** (`src/count_tubes.py`, `src/count_planks.py`) are kept for the record. Hough circles with a lattice filter count clean front-view tubes well (92 vs 91 on the reference screenshot) but fail on oblique stacks. The 1-D plank layer counter over-counted by 1 to 3 per column. Neither is used for the results.

3. Inference time

Hardware	Time per image
Colab T4 GPU	about 15 ms
2 CPU cores (cloud container)	about 70 to 80 ms

YOLOv8n at 512 px, including pre- and post-processing, after warm-up. iPhone speed has not been measured.

4. Limitations

- The ROIs, thresholds (0.5 / 0.5 / aspect 6) and the ROI for image 5 were chosen **after looking at results on these same 5 images**, so the numbers are optimistic. A fair evaluation needs real annotated images that are not only the test images.
- The reference counts are my visual estimates, not ground truth (Needs of real and huge dataset).

5. How to make it robust

5. Annotate 30 to 50 real stack photos in CVAT (one box per plank end, labels `plank_end` and `tube_end`). Fine-tune from the synthetic weights, mixing synthetic and real images (about 70/30).
6. Replace the manual box with a draggable quad plus perspective rectification, or a small segmentation of the selected stack.
7. Augment real images (perspective, exposure, glare, JPEG) and copy-paste real stacks onto real backgrounds.
8. Evaluate by count error on whole held-out images: mean absolute error and share within +/-1 plank.
9. Add in-app correction (tap to add or remove a plank) and log corrections for retraining.

7. iOS and Core ML

- **Export.** `yolo export model=best.pt format=coreml nms=True half=True imgsz=512` produces `best.mlpackage`. It takes a 512x512 image plus `confidenceThreshold` and `iouThreshold` and returns `confidence` and `coordinates`. The export succeeded on Colab. I have **not run the Core ML model on a Mac or iPhone**, so on-device accuracy and speed are untested. An ONNX export is also provided.
- **App flow.** Capture, user drags a box or quad around the stack, rectify, run `VNCoreMLRequest`, keep boxes inside the region and drop very wide boxes, overlay numbered markers, tap to correct.
- **Real conditions to handle.** Dusk, glare and shadows, angles up to about 45 degrees, rust or wet steel, partly hidden or tilted planks, mixed plank sizes, tubes with end caps, cluttered warehouses, and very large photos (resize to 512 px for the model but keep the original for display).
- **Quality gates.** Show a retake prompt when many detections have low confidence, and always show the markers so the user can verify the count.
- **Speed.** A small detector in FP16 on a recent iPhone is typically tens of milliseconds. Measure on device.

8. How to reproduce

Colab (recommended): open `notebooks/colab_scaffold_counter.ipynb`, select a T4 GPU and run all cells. It generates the data, trains, counts, and exports Core ML and ONNX.

Local:

```
pip install -r requirements.txt
python src/synth_data.py --out data/synthetic --train 1000 --val 150 --size 512 --seed 1
yolo detect train data=data/synthetic/dataset.yaml model=yolov8n.pt imgsz=512 epochs=30 batch=32
name=synth
python src/count_in_roi.py runs/detect/synth/weights/best.pt path/to/images rois.json
python app/app.py
```

`rois.json` maps each image name to `[x0, y0, x1, y1]` in pixels. The 5 test images are confidential and are not part of the code repository.

Done by Christian Ishimwe



Christian Ishimwe
https://github.com/ChristianIshimwe7
Gisenyi, Rwanda
+250 787 563 648
ishimwechris765@gmail.com